Using Artificial Intelligence For Deciphering Large Collections Of Historical Documents in Dubrovnik State Archive

Ivona Zakarija
Department of Electrical Engineering and Computing
University of Dubrovnik
Dubrovnik, Croatia
ivona.zakarija@unidu.hr

Matej Kajinić
Department of Electrical Engineering and Computing
University of Dubrovnik
Dubrovnik, Croatia
matejkajinic@gmail.com

Vlaho Čerjan
Department of Electrical Engineering and Computing
University of Dubrovnik
Dubrovnik, Croatia
vlahocerjan9@gmail.com

Mario Tucak
Department of Electrical Engineering and Computing
University of Dubrovnik
Dubrovnik, Croatia
mario.tucak@stud.unidu.hr

Abstract 2

Introduction 3

Experiments and results 5

Conclusion 11

Acknowledgment 12

References 13

Abstract

Keywords: Artificial Intelligence, Deep learning, Handwritten text recognition, Tesseract, Optical character recognition (OCR) , history documents

Introduction

3A-D20 (An Archive Automation-Decoder 2.0) is a research project that aims to develop a set of tools for deciphering large collections of historical documents for Dubrovnik State Archive. In this paper, we are showing a first step - conceptual solution for OCR (Optical Character Recognition) with HTR (Handwritten Text Recognition) capabilities.

Concept solution is based around Tesseract 4.1.1. - OCR engine and PyTesseract 0.3.4, with Python 3.7.1, OpenCV-Python 4.2.0, Leptonica 1.79.0, Pandas 1.0.4, NumPy 1.18.0 and Matplotlib 3.2.1.

Tesseract 4 has more than 100 pre-trained data files [6], so we decided to use 5 that we thought would suit us the most. Selected languages are shown at Table 1 [1].

Table 1: Selected languages

Langcode	***Language***
eng	English
enm	English, Middle (1100-1500)
ita	Italian
ita_old	Italian - Old
lat	Latin

Image processing was done with the following algorithms, shown at Table 2.

Table 2: Image processing algorithms

Id	***Name***
1	Original grayscale
2	Threshold
3	Deskewed

Algorithms, known as OEM (OCR Engine Mode), used for the project are shown at Table 3 [2].

Table 3: Tesseract algorithms

OEM N	***Name***
0	Original Tesseract
1	Neural nets LSTM (Long short-term memory)
2	Tesseract + LSTM
3	Default, based on what is available

For our work presented in this paper, we have drawn an idea from the project "In Codice Ratio" [3]. We wanted to see if it's possible to use existing open source projects, with minimal resources and time, to create a fast and easy to use concept solution for Dubrovnik State Archive.

The current problem with archives is physical space for all papers, how to store them securely for centuries to come, priceless value, availability, searchability and translation. We are hoping to speed up the process of digitization and making them as available as possible. We are showing promising results without too many resources, using open source projects.

Experiments and results

In order to verify our approach, the proposed concept solution was tested on real data with two sets of tests. We used small parts of *"LIBER STATUTORUM CIVITATIS RAGUSII"* or Statue of Dubrovnik from 1272, written in Latin, with a small part of the selected text is shown in Figure 1 [4]. All input data are selected and copied manually, word by word, as shown at Figure 2. The statue has a detailed translation so we could confirm our results at any moment with ground truth (expected results). Total number of selected words are 112, from part of one selected page, with different lengths from 2 up to 10 letters. Some words are used multiple times.

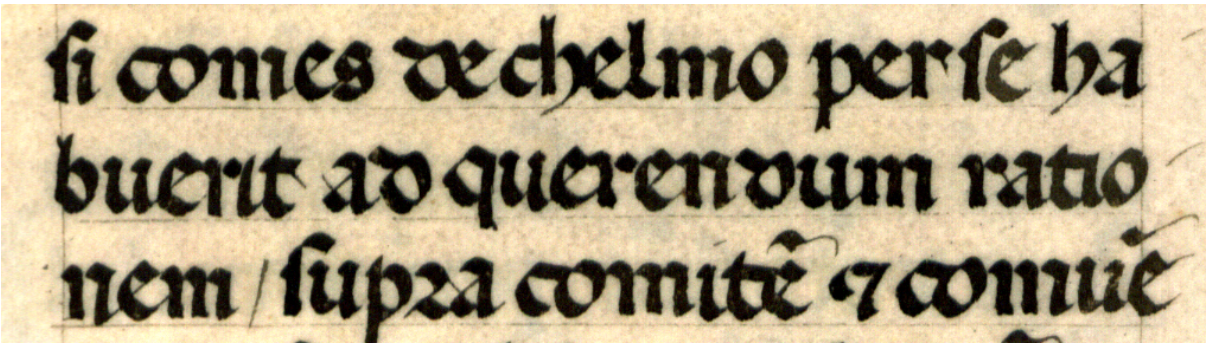

Figure 1: Sample text from the Statue of Dubrovnik

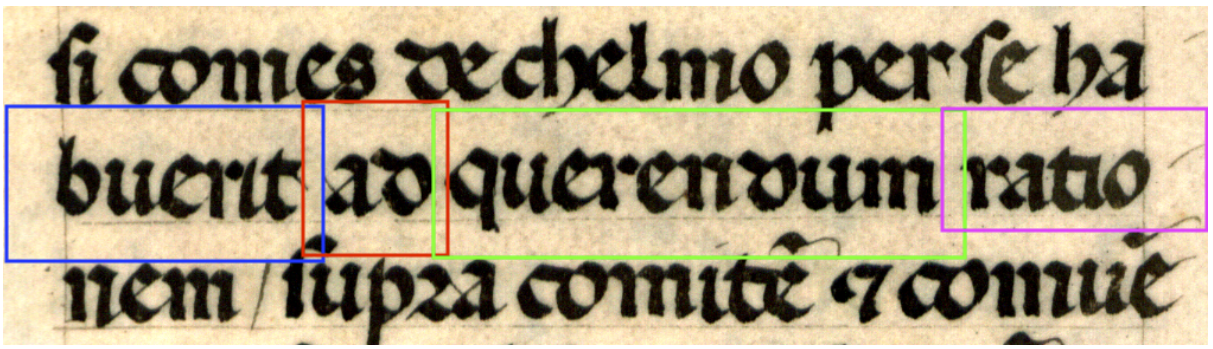

Figure 2: Example of selecting and cropping words from the Statue of Dubrovnik

Step by step process of the test is shown in Figure 3.

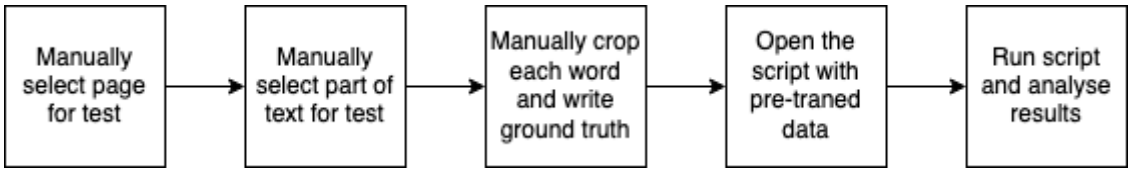

Figure 3: Step by step process for tests

For the first test, we wanted to see the differences between languages and image processing algorithms. For every word, there are 15 output results (5 languages with 3 image processing), totaling 1680 rows of results. Only a small portion of the results is shown, due to lack of space.

After initializing the Python script, plots are automatically exported to a specific folder, and results are written in the .CSV (comma-separated value) file for all input data. The whole process is finished in a few minutes.

To confirm our tests, we had to establish criteria. Our criteria was for every input, at least one out of 15 outputs have to be correctly translated to be accepted. Due to different writing in scans and ground truth, we decided to accept all results in a non case sensitive style. With that we could calculate the accuracy of the solution. Accuracy is one metric for evaluating classification models. Formally, accuracy has the following definition:

$$Accuracy = \frac{Number\ of\ correct\ predictions}{Total\ number\ of\ predictions}$$

[5].

From a total 112 words, first results are 46 correctly translated words, which is 41,07% accuracy.

Some words, like *"chelmo"* shown in Figure 4, are correct. Others, like the word *"fuerint"* shown in Figure 6, are having exact numbers of letters but have one letter wrong in translation.

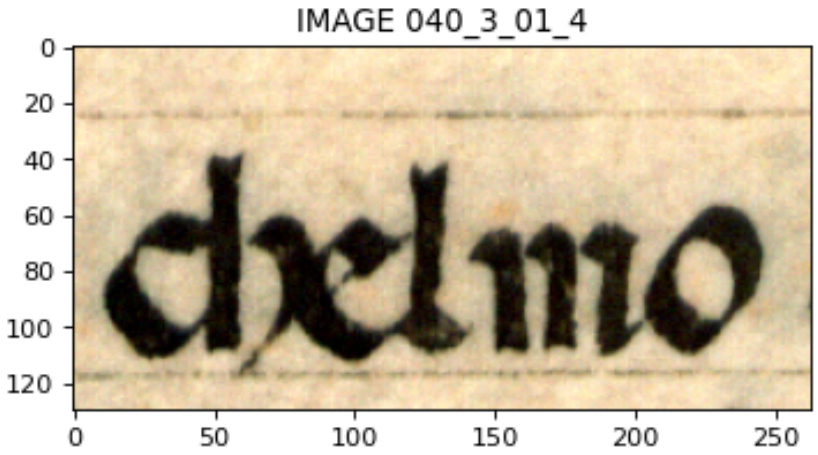

Figure 4: Latin word *"chelmo"* from the Statue of Dubrovnik

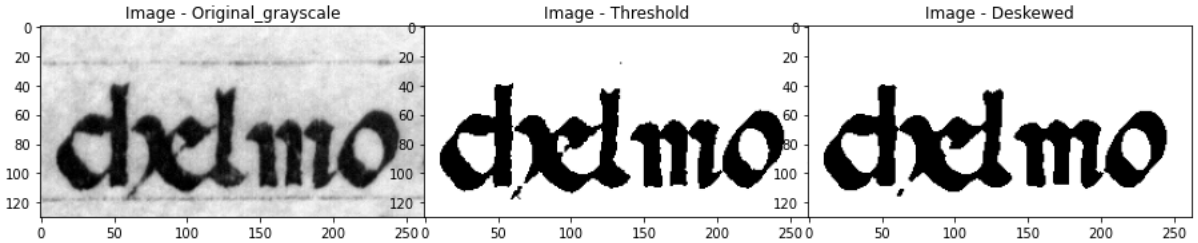

Figure 5: Latin word *"chelmo"* after image processing

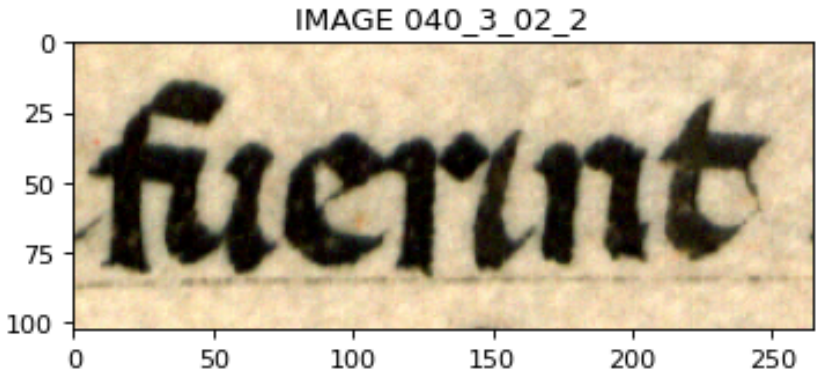

Figure 6: Latin word *"fuerint"* from the Statue of Dubrovnik

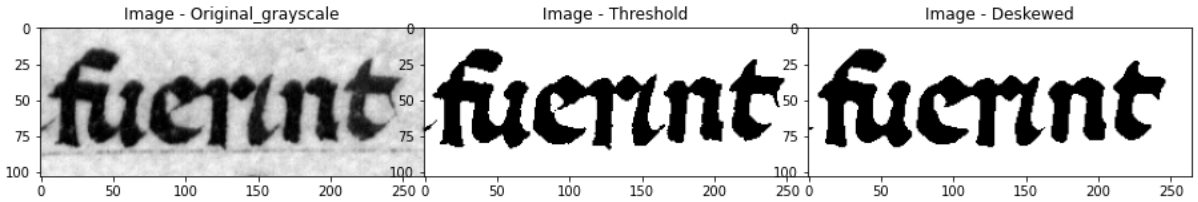

Figure 7: Latin word *"fuerint"* after image processing

If we look at all results, then we have 330 correct outputs out of 1680, due to 15 outputs for every input. For that 330, distribution between languages is shown in Figure 7, having the best results with *"enm"*.

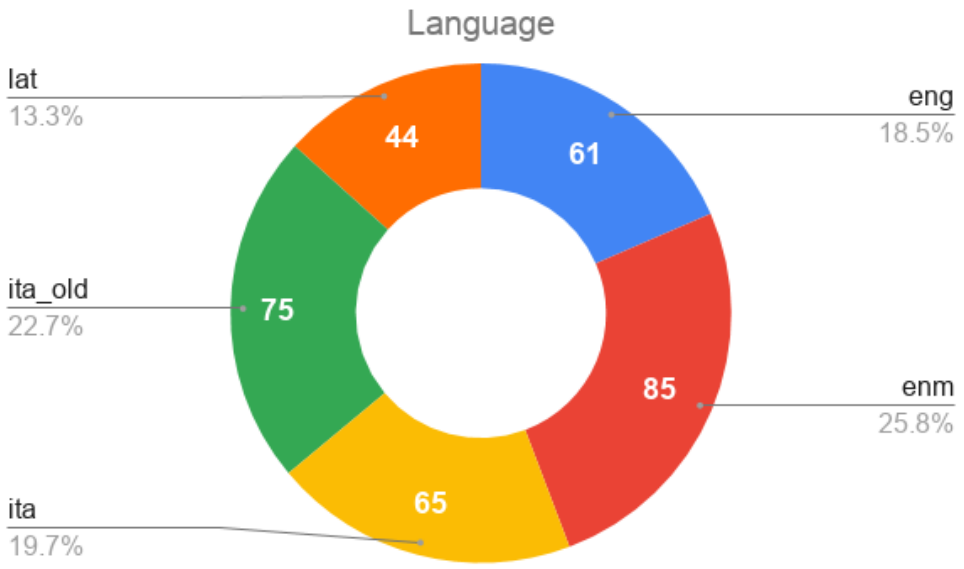

Figure 7: Distribution between languages

For image processing algorithms, best results were achieved with *"Deskewed"* algorithm, as shown in Figure 8.

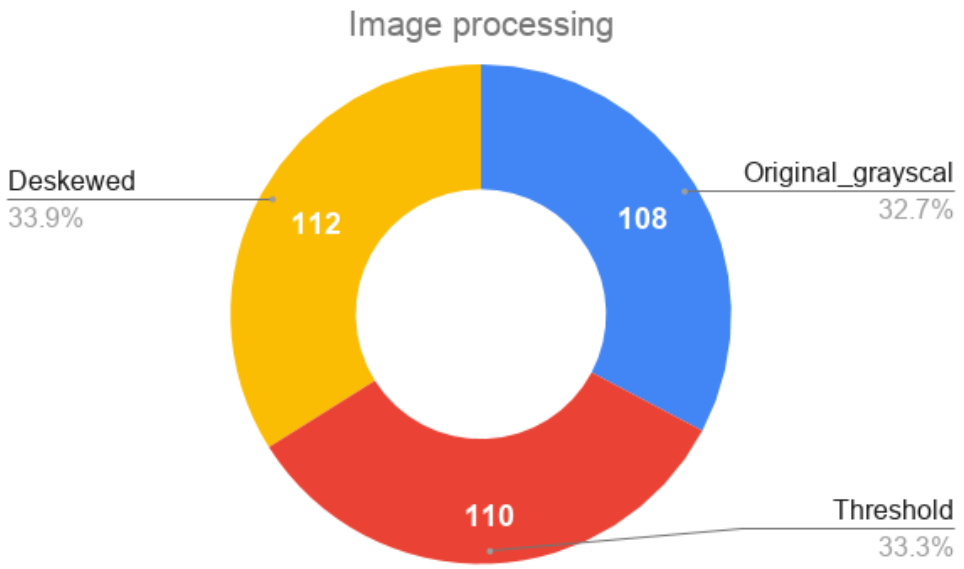

Figure 8: Distribution between image processing algorithms

If we look at the best combination of language + image processing, the results are with *"enm + Deskewed"*, as shown in Figure 9.

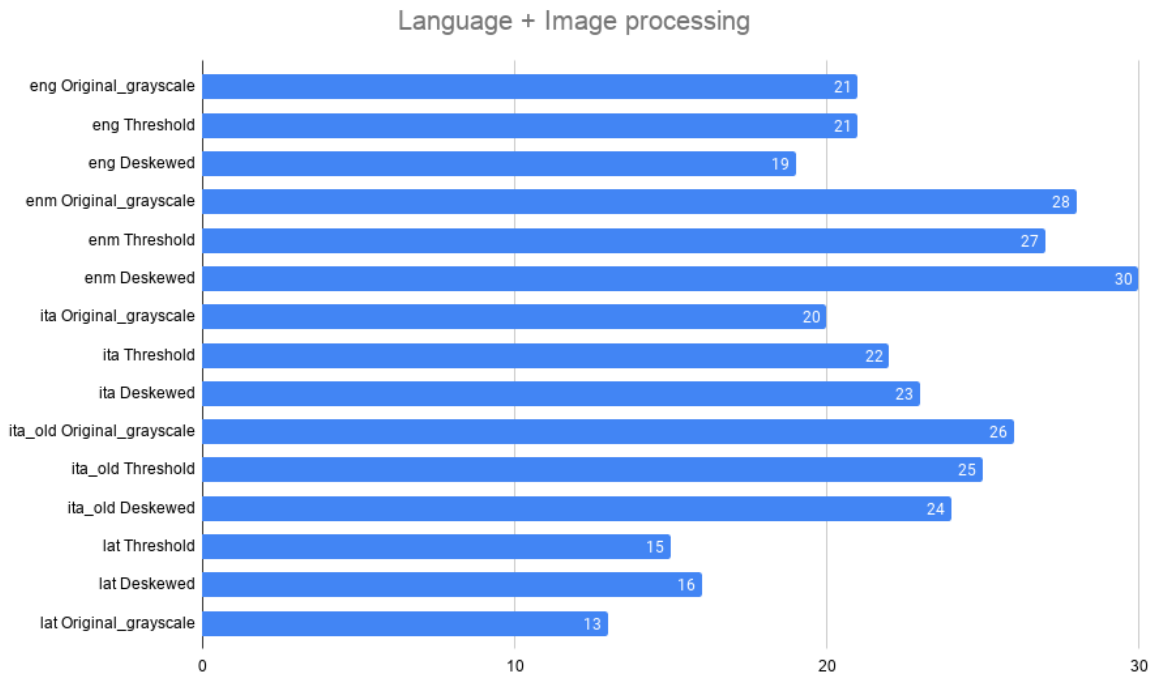

Figure 9: Distribution between language + image processing algorithms

For the second test, we wanted to see the differences between all OEM algorithms, shown in Table 3. Inputs are the same, 112 pictures, and after the first test, we used *"enm"* language with *"Deskewed"* image processing algorithm. For each input, we have 4 output results, totaling 448 rows of results. The same rule of accuracy is applied in this test, scoring 32 correctly translated words out of 112, which is 28,57% accurate. If we look at all results, then we have 102 correct outputs out of 448, due to 4 outputs for every input.

Best algorithm, for this environment, was Tesseract + LSTM (OEM N=2), as shown at Figure 10. That algorithm was also Default option inside Tesseract (OEM N=3).

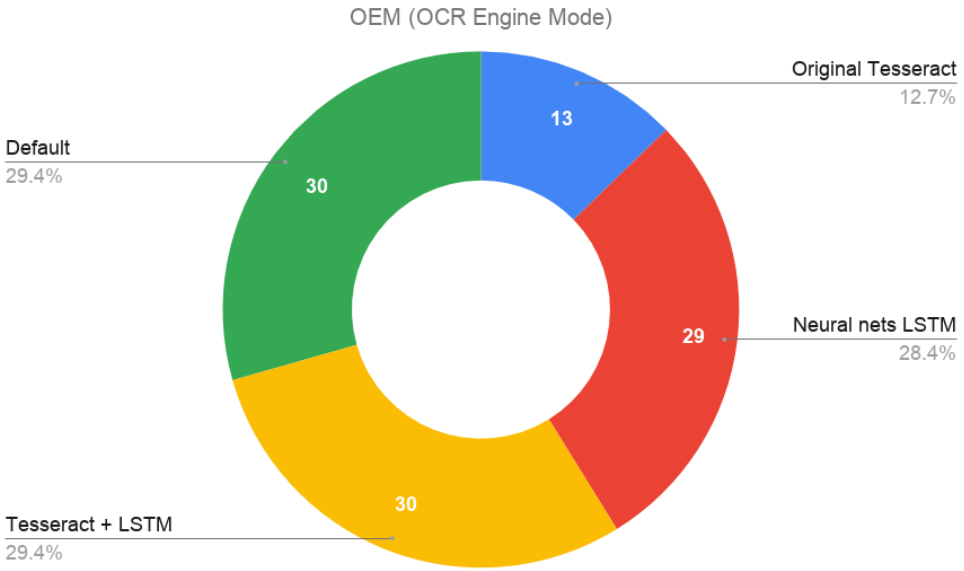

Figure 10: Distribution between OEM

Conclusion

There are many imperfections in huge piles of documents due to age and archiving, but also different styles of writing, typos, and also there are some specific words that are not used often. We found multiple cases where some word is written in two lines, but without any special mark, and also at the end of sentences without comma. All this makes impossible task for only one technology to provide a high level of correct feedback.

Tesseract is a fast and reliable OCR engine for building a set of tools around it. We see a solution in combination with different technologies in a subset of AI (Artificial Intelligence). Automating scanning documents and exporting with defined nomenclature, automatic image processing for word separation, adding artificial buffers and removing noise or imperfections from the picture, and finally OCR with specific NLP and LSTM for a specific region and time. With specific dataset, we could use all OEM's provided with Tesseract and find what works the best. In the final stage, we are planning to develop GUI (Graphical User Interface) for application.

Our main goal was showing how promising results can be achieved with no specific resources, small team, fast and without huge computing power. The solution is easy to use and easy to transfer on any platform.

This solution could provide a powerful tool for people that do this everyday, for speeding up processes and eliminating errors that are occurring due to highly repetitive and sensitive work.

Acknowledgment

We thank Dubrovnik State Archive for their support with data and information.

References

[1] “Tessdoc.” Tessdoc, https://tesseract-ocr.github.io/tessdoc/Data-Files.html. Accessed 19 June 2020.

[2] “Tesseract-Ocr/Tesseract.” GitHub, https://github.com/tesseract-ocr/tesseract. Accessed 22 June 2020.

[3] *In Codice Ratio*. http://www.inf.uniroma3.it/db/icr/index.html. Accessed 22 June 2020.

[4] Statut Grada Dubrovnika, Državni arhiv u Dubrovniku, Dubrovnik 2002.

[5] “Classification: Accuracy | Machine Learning Crash Course.” Google Developers, https://developers.google.com/machine-learning/crash-course/classification/accuracy. Accessed 1 Aug. 2020.

[6] *Tesseract-Ocr/Tessdata*. 2015. tesseract-ocr, 2020. *GitHub*, https://github.com/tesseract-ocr/tessdata. Accessed 1 Aug. 2020.